

Sistema Distribuido de Reconocimiento de Objetos con IA (CNN) y Consenso Raft

Examen Final – CC4P1 Programación Concurrente y Distribuida

Melissa Iman Noriega

Junior Ortega

Escuela de Ciencias de la Computación – Facultad de Ciencias – UNI

2026-I

Resumen

Construimos un sistema distribuido que reconoce figuras en tres cámaras con una **CNN escrita desde cero en NumPy** y guarda cada detección en un **registro replicado con Raft**. El cúmulo es heterogéneo: un nodo en **Java**, uno en **Python** y uno en **Go**, cada uno con su implementación de Raft hablando el mismo protocolo de texto sobre **sockets TCP nativos**. No usamos frameworks de mensajería ni librerías de red de terceros: solo la biblioteca base de cada lenguaje (NumPy se permite para la CNN). El cúmulo elige líder, la CNN clasifica en las 3 cámaras (un hilo por cámara) con 88–100 % de acierto por cámara, y los tres nodos aplican las mismas 75 detecciones en el mismo orden. Probamos el failover matando al líder (Python): Go es reelegido en el siguiente *term* y el registro sobrevive consistente. La CNN alcanza 97.2 % en test y su entrenamiento distribuido (data-parallel con **multiprocessing**, 4 *workers*) baja el tiempo de 11.0s a 4.7s sin perder accuracy.

1. Datos del proyecto y reparto

El sistema tiene tres piezas: un *Servidor de Testeo* en Python que corre la CNN sobre las cámaras, un *cúmulo Raft* de tres nodos que mantiene el registro replicado, y un *Cliente Vigilante* en Java que consulta ese registro. La Tabla 1 resume quién hizo qué.

Integrante	Lenguaje	Componentes
Melissa Iman	Java	Núcleo Raft, entrenamiento distribuido, Cliente Vigilante (UI)
Junior Ortega	Go	Nodo Raft, máquina de estado del registro
(parte Python)	Python	CNN desde cero, Servidor de Testeo, nodo Raft

Cuadro 1: Reparto del trabajo por lenguaje.

2. Enunciado y cumplimiento

El enunciado pedía un sistema distribuido de reconocimiento con IA, consenso sin librerías externas y comunicación por sockets. La Tabla 2 lista cada requisito, su estado y dónde se resuelve.

3. Arquitectura

El flujo va de las cámaras al Servidor de Testeo, del servidor al cúmulo Raft como escrituras en el registro, y del cúmulo al Cliente Vigilante que lee. Cada detección que produce la CNN se manda al líder del cúmulo; el líder la replica a los seguidores y, cuando hay mayoría, se aplica a la máquina de estado (el registro). La Figura 1 lo muestra.

Requisito	Estado	Dónde
3 lenguajes, un nodo cada uno	✓	Java, Python, Go (§3)
Solo sockets nativos, sin websocket / socketio / frameworks / MQ	✓	TCP crudo en los 3 nodos
Consenso Raft implementado a mano	✓	Elección + replicación (§4)
Hilos / goroutines / mutex	✓	Un hilo por cámara; mutex por estado Raft
Solo librerías base (NumPy para la CNN)	✓	CNN en NumPy; red en socket/net
Reconocimiento con IA (CNN)	✓	97.2 % en test (§6)
Entrenamiento distribuido	✓	multiprocessing, 4 workers (§6)
Registro replicado y consistente	✓	75 detecciones, mismo orden (§7)
Tolerancia a fallos (failover)	✓	Se mata al líder, Go es reelegido (§7)
Despliegue local sin internet, LAN/WIFI	parcial	Nodos escuchan en 0.0.0.0; demo multi-PC pendiente

Cuadro 2: Requisitos del enunciado y estado.

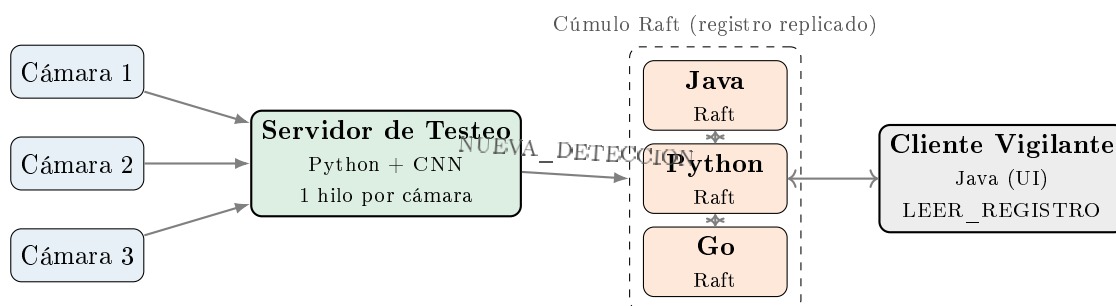


Figura 1: Arquitectura: 3 cámaras → Servidor de Testeo (Python+CNN) → cúmulo Raft heterogéneo {Java, Python, Go} → Cliente Vigilante. Toda escritura pasa por el líder y se replica por mayoría.

Concurrencia. El Servidor de Testeo levanta un hilo por cámara, de modo que las tres corren la CNN en paralelo. Cada nodo Raft protege su estado (term actual, voto, log, `commitIndex`) con un mutex, porque el hilo que recibe mensajes de red y el que corre el temporizador de elección tocan las mismas variables. En Go esto es una goroutine por conexión más un mutex sobre la máquina de estado; en Java y Python es un hilo por conexión con `synchronized` / `Lock`.

4. Consenso Raft

La implementación sigue el paper de Ongaro y Ousterhout, el mismo material del curso. Cada nodo está en uno de tres estados: **Seguidor**, **Candidato** o **Líder**.

Elección de líder. Un seguidor que no recibe latidos del líder dentro de su *timeout* (aleatorio entre 150 y 300 ms) pasa a Candidato, incrementa su `term`, vota por sí mismo y envía `REQUEST_VOTE` a los demás. El timeout aleatorio evita que los tres nodos se postulen a la vez. Un nodo concede el voto solo si el candidato está al menos tan actualizado como él (restricción de elección: se comparan `lastLogTerm` y `lastLogIndex`). Con mayoría de votos el candidato se vuelve Líder y empieza a mandar latidos.

Replicación del log. El líder recibe cada `NUEVA_DETECCION`, la agrega a su log y la propaga con `APPEND_ENTRIES`. Cada mensaje lleva `prevLogIndex` y `prevLogTerm`; el seguidor solo acepta si esos campos calzan con su propio log (*Log Matching*). Si no calzan, el líder retrocede el `nextIndex` de ese seguidor y reintenta hasta encontrar el punto común, sobrescribiendo lo que diverja.

Seguridad y commit. Una entrada se considera *committed* cuando está replicada en la mayoría. Respetamos la regla del paper: el líder solo cuenta como committed las entradas de **su term actual**; las de terms anteriores se confirman de forma indirecta al confirmarse una posterior. Recién cuando una entrada está committed se aplica a la máquina de estado (el registro). Así los tres nodos aplican las mismas detecciones en el mismo orden.

El siguiente extracto muestra cómo un nodo decide conceder o no su voto al recibir `REQUEST_VOTE`, con la restricción de elección que compara quién tiene el log más actualizado.

```

1 al recibir REQUEST_VOTE(term, candId, lastLogIdx, lastLogTerm):
2   con mutex bloqueado:
3     si term < termActual: # candidato atrasado
4       responder VOTE(termActual, granted=0)
5     si term > termActual: # hay un term mas nuevo
6       termActual = term; votadoPor = null; volverASeguidor()
7     # restriccion de eleccion: el candidato debe estar al dia
8     alDia = (lastLogTerm > miUltimoTerm) o
9             (lastLogTerm == miUltimoTerm y lastLogIdx >= miUltimoIdx)
10    si (votadoPor == null o votadoPor == candId) y alDia:
11      votadoPor = candId; reiniciarTimeout()
12      responder VOTE(termActual, granted=1)
13    sino:
14      responder VOTE(termActual, granted=0)

```

Listing 1: Lógica de voto al recibir `REQUEST_VOTE` (misma en los 3 nodos).

5. Protocolo de texto

Todos los mensajes son una línea de texto terminada en salto de línea, con campos separados por `|`. Los tres lenguajes hablan exactamente el mismo formato, lo que hace posible un cúmulo heterogéneo sobre TCP crudo. Hay dos grupos: los mensajes de Raft y los del registro/cliente.

```

1 # --- Raft ---
2 REQUEST_VOTE|term|candidateId|lastLogIndex|lastLogTerm
3 VOTE|term|granted
4 APPEND|term|leaderId|prevLogIndex|prevLogTerm|commitIndex|entries
5 APPEND_OK|term|success|matchIndex
6
7 # --- Registro y cliente ---
8 NUEVA_DETECCION|tipo,imagenRef,fechaHora,camara
9 LEER_REGISTRO
10 REGISTRO|det1;det2;...
11 REDIRECT|host:puerto
12 OK

```

Listing 2: Protocolo de texto compartido (Raft + registro).

Un detalle: si un cliente o el Servidor de Testeo le escribe a un seguidor, este responde `REDIRECT` con `host:puerto` apuntando al líder actual, en vez de rechazar la escritura. Así el Servidor de Testeo siempre termina escribiendo en el líder aunque cambie tras una elección. El campo `entries` de `APPEND` lleva cero o más detecciones serializadas; cuando va vacío, el mensaje funciona como latido (*heartbeat*) que mantiene vivo al líder y reinicia el temporizador de elección de cada seguidor.

6. Modelo de IA: CNN desde cero

La CNN clasifica figuras de 28×28 en 5 clases: círculo, cuadrado, triángulo, cruz y línea. Está escrita a mano en NumPy, sin librerías de deep learning. La arquitectura es:

Conv $8@3 \times 3$ + ReLU + MaxPool $\rightarrow$ Conv $16@3 \times 3$ + ReLU + MaxPool $\rightarrow$ Flatten $\rightarrow$ Densa $\rightarrow$ Softmax.

El *forward* y el *backward* (backpropagation) están implementados a mano. La convolución usa `im2col`: se reordena la imagen en columnas para que la convolución quede como una sola multiplicación de matrices, que es mucho más rápida que iterar píxel por píxel. Los pesos entrenados se guardan en un `.npz` y el Servidor de Testeo los carga al arrancar. En test la red llega a **97.2 % de accuracy**.

Entrenamiento distribuido. Entrenamos en paralelo con `multiprocessing` en esquema *data-parallel*: se parte el lote entre los *workers*, cada uno calcula el gradiente sobre su parte y se promedian los gradientes (all-reduce) antes de actualizar los pesos. El resultado es el mismo modelo (97.2 %) en menos tiempo. La Tabla 3 compara.

Entrenamiento	Workers	Tiempo	Accuracy test
Secuencial	1	11.0 s	97.2 %
Distribuido (data-parallel)	4	4.7 s	97.2 %

Cuadro 3: Entrenamiento secuencial vs distribuido. Misma accuracy, $\sim 2.3\times$ más rápido con 4 workers.

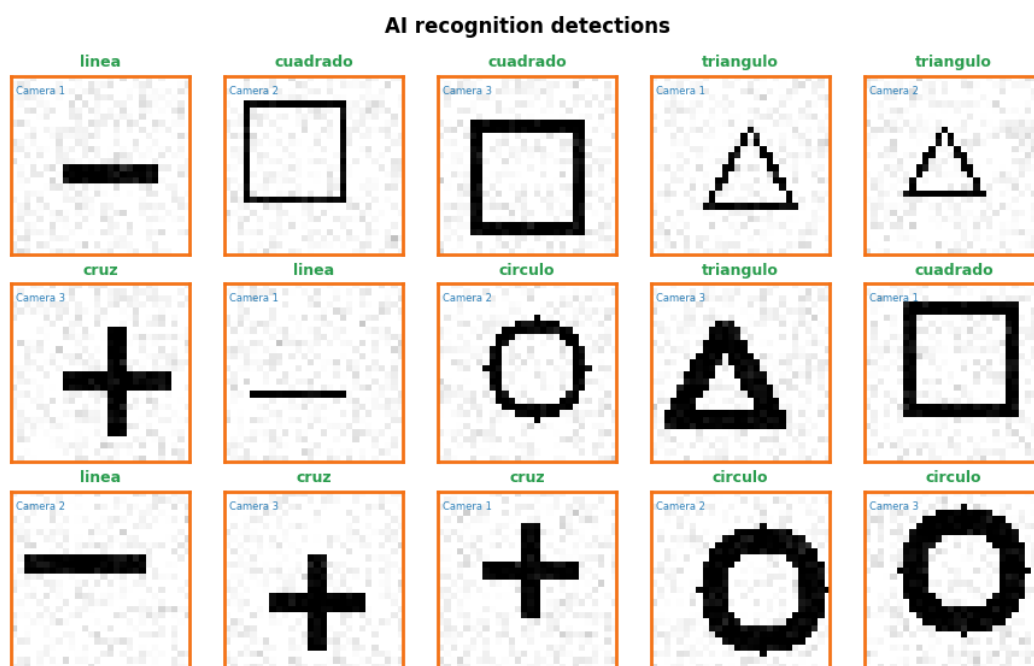


Figura 2: Detecciones reconocidas por la CNN (etiqueta en verde, cámara en azul).

7. Pruebas end-to-end y tolerancia a fallos

Levantamos el cúmulo con los tres nodos (Java + Python + Go) y el Servidor de Testeo con las 3 cámaras. El sistema funciona de punta a punta:

- El cúmulo heterogéneo **elige líder** sin intervención.
- La CNN **reconoce en las 3 cámaras** en paralelo (un hilo por cámara), con **88–100 % de acierto** por cámara.
- Los tres nodos aplican las **mismas 75 detecciones en el mismo orden**: el registro queda idéntico en Java, Python y Go.

Consistencia del registro. La forma de verificar que la réplica quedó bien fue comparar el registro de los tres nodos al final de la corrida: pedimos `LEER_REGISTRO` a cada uno y comparamos las 75 detecciones. Las tres respuestas `REGISTRO|...` salieron idénticas, con las detecciones en el mismo orden. Esto confirma que Raft no solo replicó el contenido, sino también el orden, que es lo que garantiza la máquina de estado determinista: aplicar el mismo log en el mismo orden lleva a los tres nodos al mismo estado.

Failover. Para probar tolerancia a fallos matamos al líder mientras el sistema seguía insertando. En esa corrida el líder era el nodo Python. Al caer, los seguidores dejan de recibir latidos, salta el timeout de elección y **Go es reelegido líder en el siguiente *term***. El registro sobrevive consistente (ninguna detección committed se pierde) y se sigue insertando con normalidad. Esto ejercita justo la parte difícil de Raft: cambiar de líder sin corromper ni reordenar el log ya confirmado.

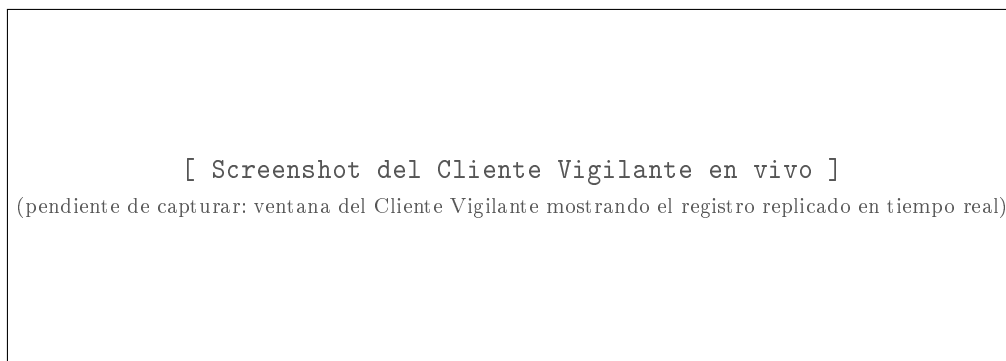


Figura 3: Cliente Vigilante mostrando el registro replicado (captura pendiente).

8. Cómo ejecutar

Todo es local, sin internet. Se entrena la CNN una vez para generar los pesos `.npz`; luego se levantan los tres nodos Raft (cada uno en su lenguaje) y por último el Servidor de Testeo y el Cliente Vigilante.

1. Entrenar la CNN (Python + NumPy) → guarda pesos en `.npz`.
2. Arrancar los 3 nodos Raft: Java, Python y Go, cada uno con su puerto. Escuchan en `0.0.0.0`, así que sirven tanto en localhost como en LAN/WIFI.
3. Arrancar el Servidor de Testeo (Python), que carga los pesos y abre un hilo por cámara.
4. Arrancar el Cliente Vigilante (Java) para ver el registro replicado.

9. Conclusiones y trabajo futuro

- Armamos un cúmulo **poliglota** (Java + Python + Go) donde los tres nodos corren su propia implementación de **Raft a mano** sobre TCP crudo, cumpliendo la restricción de no usar frameworks ni MQ.
- La **CNN desde cero en NumPy** clasifica las 5 figuras con 97.2 % en test, y su **entrenamiento distribuido** baja el tiempo de 11.0 s a 4.7 s con 4 workers sin perder accuracy.
- El sistema es consistente de punta a punta: 75 detecciones aplicadas en el mismo orden por los tres nodos, y **failover** verificado (muere el líder Python, Go es reelegido, el registro sobrevive).
- **Trabajo futuro / despliegue físico:** correr los nodos en varias PCs sobre LAN/WIFI y en 2 sistemas operativos distintos. El código ya escucha en `0.0.0.0`, así que la demo multi-PC es cuestión de levantar cada nodo en su máquina; falta la corrida física en clase.